AD PATEL INSTITUTE OF TECHNOLOGY

CAR RENTAL SYSTEM

Database Management System — Project Report

Submitted by

Enrollment Number	Team Member	Suggested Role (confirm before submission)
12502080601050	Kaushal Akbari	Database design and SQL implementation
12502080601051	Kavya Parmar	Requirements and documentation
12502080601023	Foram Prajapati	Sample data and query testing
12502080601028	Drashti Prajapati	ER diagram, testing evidence, presentation

Group Number: [INSERT GROUP NUMBER] **Submitted To:** [INSERT FACULTY NAME]

Department: Information Technology

Institute: Ad Patel Institute of Technology

Academic Year: [INSERT ACADEMIC YEAR] **Submission Date:** [INSERT DATE]

Prepared in accordance with the supplied DBMS Project Guide, Topics & Submission Guidelines.

CERTIFICATE

This is to certify that the project report titled **“Car Rental System”** has been prepared by the following students of the Department of Information Technology as part of the Database Management System project work.

No.	Name	Enrollment Number
1	Kaushal Akbari	12502080601050
2	Kavya Parmar	12502080601051
3	Foram Prajapati	12502080601023
4	Drashti Prajapati	12502080601028

The report describes the proposed database design, relational schema, SQL operations, and application architecture. Details that require evidence from the final implementation are explicitly marked for verification.

Project Guide / Faculty	Head of Department	Date
Signature: _____	Signature: _____	_____

Note: Obtain the required signatures and confirm the institute's prescribed certificate wording before final submission.

ACKNOWLEDGEMENT

We express our sincere gratitude to our faculty guide and the Department of Information Technology for providing the opportunity to work on this Database Management System project. The project gives us an opportunity to apply relational database concepts to a practical record-management scenario.

We also acknowledge the learning resources and software tools used during the development process, including MySQL, MySQL Workbench, Visual Studio Code, Node.js, Express.js, and a modern web browser. These tools support database design, SQL development, backend programming, and interface testing.

The team members contributed to planning, documentation, database design, query preparation, and testing activities. The role assignments shown in this report are suggestions and should be updated to reflect each member's actual contribution.

DECLARATION

We declare that this report is prepared for academic evaluation of the Car Rental System DBMS project. Any sample SQL or conceptual diagrams are identified as illustrative where the final source code or database schema was not supplied for verification.

Team Members' Signatures

Kaushal Akbari	Kavya Parmar	Foram Prajapati	Drashti Prajapati
_____	_____	_____	_____

ABSTRACT

The Car Rental System is a database-oriented project intended to organize information related to vehicles, customers, rental transactions, and payments. A relational database can store these records in linked tables and use primary keys, foreign keys, and constraints to preserve relationships and reduce unnecessary repetition. The project is planned around a browser-based interface built with HTML5, CSS3, and JavaScript, a Node.js backend using Express.js, and a MySQL database named **CarRentalDB**. The **mysql2** driver provides the backend connection to MySQL. MySQL Workbench is used separately for database administration, schema inspection, and SQL development.

The report follows the structure recommended in the supplied AD Patel Institute of Technology DBMS project guidelines. It covers the project background and requirements, business rules, entity-relationship design, relational schema mapping, normalization principles, SQL DDL and DML examples, interface/database evidence, testing, conclusion, limitations, and future improvements. The four principal entities are Vehicles, Customers, Rentals, and Payments. Since the final SQL script, application screenshots, and verified test outputs were not included with the supplied materials, schema details and SQL examples in this report are presented as a proposed design and must be checked against the implementation before submission. No test result is represented as completed without evidence.

Keywords: DBMS, Car Rental System, MySQL, relational schema, ER diagram, SQL, normalization, Node.js, Express.js.

CHAPTER 1 — INTRODUCTION & SYSTEM REQUIREMENTS

1.1 Background

A car rental operation needs to maintain vehicle details, customer information, rental periods, vehicle status, and payment records. When these records are stored separately or updated manually, searching for related information and maintaining consistency can become difficult. A database-backed system provides a structured way to store related information and retrieve it through queries.

1.2 Scope of the Project

• Maintain vehicle information such as registration number, make/model, type, daily rate, and availability.

• Store customer contact and driving-licence details.

• Record rental transactions that connect a customer with a vehicle and a rental period.

• Maintain payment records associated with rentals.

• Use SQL queries to retrieve, join, and summarize related data.

• Provide a browser-based interface connected to a backend API where that workflow is implemented and tested.

1.3 Project Objectives

• Organize related data into separate relational tables.

• Identify records with primary keys and link them with foreign keys.

• Apply uniqueness and required-field constraints where appropriate.

• Demonstrate DDL, DML, SELECT, JOIN, and GROUP BY operations.

• Document implementation status honestly using screenshots and verified test results.

1.4 Software Requirements

Component	Technology / purpose
Frontend	HTML5, CSS3, JavaScript
Runtime / backend	Node.js with Express.js
Database	MySQL Server; database name: CarRentalDB
Database driver	mysql2 for Node.js
Administration / SQL tool	MySQL Workbench
Editor / client	Visual Studio Code and a modern web browser

CHAPTER 1 — REQUIREMENTS ANALYSIS

1.5 Hardware Requirements

The project can be developed on a typical student development computer capable of running a code editor, Node.js, MySQL Server, MySQL Workbench, and a web browser. Exact processor, memory, storage, and operating-system build should be documented from the computer actually used; this report does not assume unverified specifications.

1.6 Problem Statement and Motivation

Manual / fragmented management	Database-based approach
Records may be stored in separate files or registers.	Related records are organized in centralized tables.
Repeated data entry can cause inconsistencies.	Keys and constraints help maintain consistent relationships.
Finding rental/customer/vehicle details may take effort.	SQL SELECT and JOIN queries retrieve connected information.
Rental periods and payments need careful tracking.	Rental and payment records can be linked for review.

These points describe the motivation for building the system. They are not measured performance claims; no quantitative improvement is asserted.

1.7 Proposed Business Rules

• Each vehicle is identified by a unique vehicle_id; registration_no should be unique if enforced by the final schema.

• Each customer is identified by customer_id; license_no should be unique if enforced by the final schema.

• Each rental references one customer and one vehicle through foreign keys.

• A customer may have multiple rental records over time.

• A vehicle may appear in multiple rental records over time; booking-conflict prevention requires additional date-overlap logic.

• Payments reference rentals. Whether one rental can have zero, one, or multiple payments depends on the actual foreign-key and application rules.

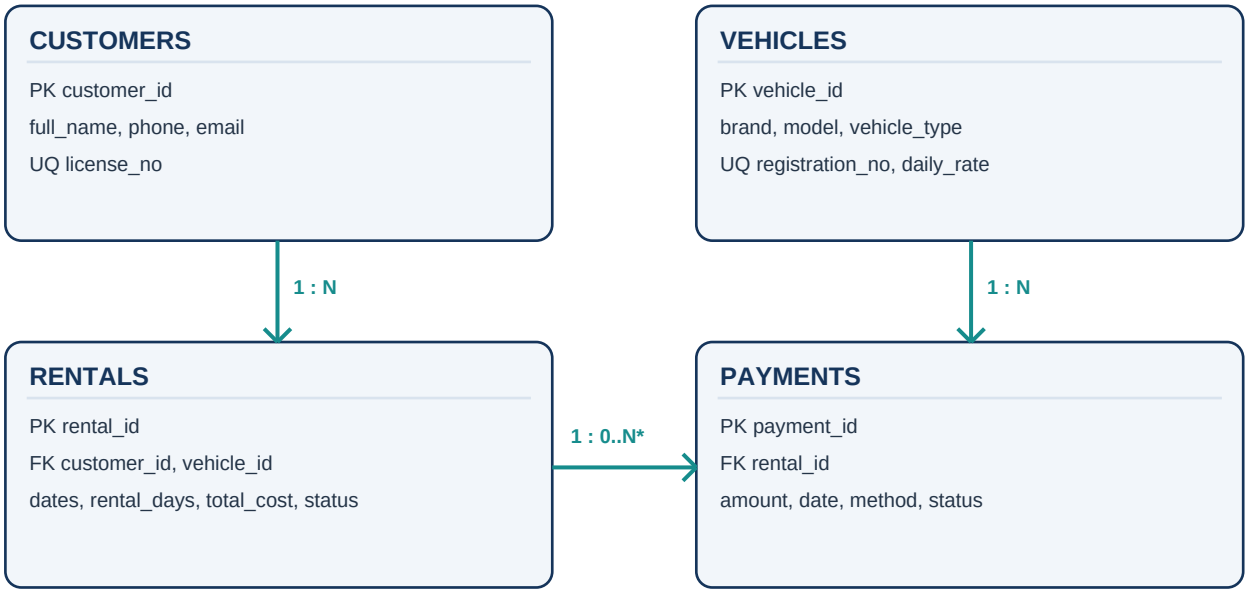
1.8 Assumptions to Verify

The final implementation should be checked for exact field names, data types, nullability, unique constraints, foreign-key actions, rental date rules, payment cardinality, and whether the frontend/backend workflows are complete. The supplied project prompt names four main entities but does not include the final SQL file.

CHAPTER 2 — DATABASE DESIGN

2.1 Entity-Relationship (ER) Diagram

The conceptual design contains four core entities: CUSTOMERS, VEHICLES, RENTALS, and PAYMENTS. Primary keys are marked with “PK” and foreign keys with “FK”. The diagram below illustrates the intended relationships; confirm every attribute and cardinality against the final SQL schema.



Conceptual mapping only: confirm exact columns and payment cardinality against the final SQL constraints.

Relationship	Intended cardinality	Reason
CUSTOMERS → RENTALS	1 : N	One customer may have multiple rental records; each rental references one customer.
VEHICLES → RENTALS	1 : N over time	One vehicle can be referenced by multiple historical rental records; date conflicts must be controlled separately.
RENTALS → PAYMENTS	1 : 0..N (verify)	A rental may have no payment yet or multiple payment records if the schema permits it.

ER notation note: the attributes and relationships are conceptual until confirmed against the database implementation. If the actual schema differs, update the diagram rather than altering the implementation description to fit the proposal.

CHAPTER 2 — RELATIONAL SCHEMA MAPPING

2.2 Proposed Relational Schema

Relation	Key fields	Other proposed attributes
VEHICLES	PK vehicle_id; UQ registration_no	brand, model, vehicle_type, daily_rate, availability
CUSTOMERS	PK customer_id; UQ license_no	full_name, phone, email
RENTALS	PK rental_id; FK customer_id; FK vehicle_id	rental_start, rental_end, rental_days, total_cost, rental_status
PAYMENTS	PK payment_id; FK rental_id	amount, payment_date, payment_method, payment_status

UQ means a uniqueness constraint. Names such as rental_start and rental_end are descriptive placeholders because exact rental date column names were not supplied. Replace them with the actual column names and data types from the final SQL script.

2.3 Key and Referential Integrity

• **Primary key:** uniquely identifies each row and must not be NULL.

• **Foreign key:** ensures a referenced customer, vehicle, or rental exists, subject to the configured constraint and actions.

• **UNIQUE:** can prevent duplicate registration numbers or driving-licence numbers.

• **NOT NULL:** requires a value for fields essential to a valid record.

• **CHECK:** can express rules such as a positive daily rate, provided the chosen MySQL version enforces the constraint.

2.4 Data Integrity Considerations

The database should prevent orphan rental records by enforcing foreign keys. Application-side validation should also check that rental end dates do not precede start dates and that the requested vehicle is available for the requested period. A simple availability flag alone does not prevent overlapping bookings unless the backend performs appropriate checks and handles concurrent requests safely.

CHAPTER 2 — NORMALIZATION ANALYSIS

2.5 First Normal Form (1NF)

A table is in 1NF when each field stores a single, atomic value and there are no repeating groups in a row. For example, customer phone and email should be represented as individual fields rather than storing a list of unrelated values in one column. If multiple phone numbers must be supported, a separate related table may be more appropriate.

2.6 Second Normal Form (2NF)

A relation is in 2NF when it is in 1NF and every non-key attribute depends on the whole of each candidate key, not only part of a composite key. The proposed tables use single-column surrogate primary keys, so partial dependency on part of the primary key is not expected. Candidate keys and any additional composite unique keys must still be reviewed in the final schema.

2.7 Third Normal Form (3NF)

A relation is in 3NF when it is in 2NF and non-key attributes do not depend transitively on a candidate key. Separating customers, vehicles, rentals, and payments helps avoid repeating customer contact details in every rental and vehicle descriptions in every transaction. The rental table stores the identifiers that link the entities rather than duplicating all their descriptive attributes.

Stage	Design check	Application to this project
1NF	Atomic fields; no repeating groups	Vehicle and customer details use individual columns.
2NF	No partial dependency on a candidate key	Proposed primary keys are single-column IDs; verify other candidate keys.
3NF	No transitive dependency among non-key attributes	Customer, vehicle, rental, and payment facts are separated by subject.

Normalization status: this is the intended design rationale, not a formal proof of the final database. A definitive 3NF claim requires the actual SQL schema and relevant functional dependencies to be reviewed.

CHAPTER 3 — IMPLEMENTATION & SQL SCRIPTS

3.1 Data Definition Language (DDL)

DDL statements define database structures and constraints. The following compact example illustrates the style of SQL expected for this project; it is not certified as an exact extract from the team's implementation.

```
CREATE DATABASE IF NOT EXISTS CarRentalDB; USE CarRentalDB; CREATE TABLE Vehicles ( vehicle_id INT AUTO_INCREMENT PRIMARY KEY, registration_no VARCHAR(20) NOT NULL UNIQUE, brand VARCHAR(50) NOT NULL, model VARCHAR(50) NOT NULL, vehicle_type VARCHAR(30), daily_rate DECIMAL(10,2) NOT NULL, availability VARCHAR(20) NOT NULL );
```

Example foreign-key relationship

```
CREATE TABLE Rentals ( rental_id INT AUTO_INCREMENT PRIMARY KEY, customer_id INT NOT NULL, vehicle_id INT NOT NULL, rental_start DATE NOT NULL, rental_end DATE NOT NULL, rental_days INT NOT NULL, total_cost DECIMAL(10,2) NOT NULL, rental_status VARCHAR(20) NOT NULL, FOREIGN KEY (customer_id) REFERENCES Customers(customer_id), FOREIGN KEY (vehicle_id) REFERENCES Vehicles(vehicle_id) );
```

The Customers table must exist before the Rentals table is created. These examples use illustrative names and should be aligned with the final schema. A CHECK constraint for positive rates or valid date ranges may be added only after compatibility and implementation are verified.

Why constraints matter

Keys and constraints improve data integrity by preventing duplicate identifiers, requiring essential values, and blocking references to nonexistent parent records. They do not replace application-level validation for booking conflicts or business rules that depend on multiple rows.

CHAPTER 3 — DML & QUERY EXAMPLES

3.2 Data Manipulation Language (DML)

DML statements add, modify, and remove records. The following are representative examples for the proposed schema and must be checked against the final field names.

INSERT example

```
INSERT INTO Vehicles (registration_no, brand, model, vehicle_type, daily_rate, availability) VALUES
('GJ00XX0000', 'ExampleBrand', 'ExampleModel', 'Hatchback', 1800.00, 'Available');
```

SELECT example — available vehicles

```
SELECT vehicle_id, brand, model, daily_rate FROM Vehicles WHERE availability = 'Available' ORDER BY
daily_rate;
```

JOIN example — rentals with customer and vehicle details

```
SELECT r.rental_id, c.full_name, v.brand, v.model, r.rental_status FROM Rentals AS r JOIN Customers AS c
ON r.customer_id = c.customer_id JOIN Vehicles AS v ON r.vehicle_id = v.vehicle_id;
```

GROUP BY example — count rentals by status

```
SELECT rental_status, COUNT(*) AS rental_count FROM Rentals GROUP BY rental_status;
```

These are query examples, not evidence of execution. The INSERT uses sample placeholder values; replace them with valid test data and capture the actual output from MySQL Workbench before submission. Views or stored procedures are not claimed as implemented because no source evidence was supplied.

CHAPTER 4 — SCREEN LAYOUTS & TEST RESULTS

4.1 Interface Screenshots

Insert cropped screenshots from the actual running application. The supplied materials did not include application screenshots, so the placeholders below are intentionally not represented as working screens.

Evidence slot	Insert actual evidence
Figure 4.1 — Main page	[INSERT ACTUAL FRONTEND SCREENSHOT HERE]
Figure 4.2 — Vehicle listing / entry	[INSERT ACTUAL SCREENSHOT HERE]
Figure 4.3 — Customer / rental / payment flow	[INSERT ACTUAL SCREENSHOT IF IMPLEMENTED]

4.2 Database Evidence

Evidence slot	What it should demonstrate
Figure 4.4 — CarRentalDB schema	Schema name and table list in MySQL Workbench.
Figure 4.5 — Sample table data	Real rows from Vehicles and Customers.
Figure 4.6 — JOIN query output	Executed query and returned records.

4.3 Testing Plan and Results

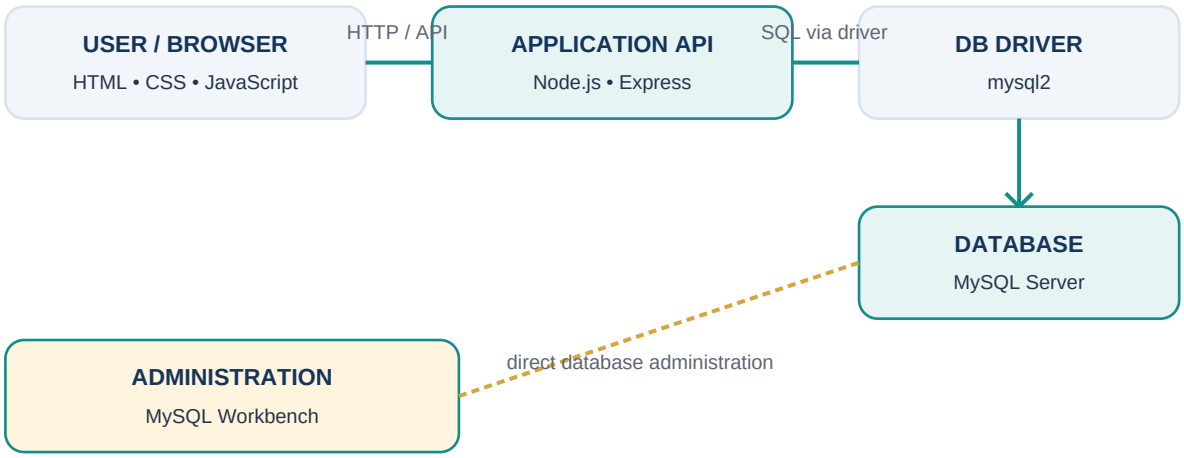
Test case	Expected result	Actual result
Insert valid customer	Record is inserted and can be retrieved.	[ENTER VERIFIED RESULT]
Insert vehicle	Vehicle row is stored with required fields.	[ENTER VERIFIED RESULT]
Retrieve available vehicles	Query returns records matching availability.	[ENTER VERIFIED RESULT]
Run customer/vehicle/rental JOIN	Related records appear with correct links.	[ENTER VERIFIED RESULT]
Foreign-key behavior	Invalid parent reference is rejected if FK is enforced.	[ENTER VERIFIED RESULT]
Rental/payment workflow	Matches implemented business rules, if supported.	[ENTER VERIFIED RESULT]

Do not mark a test as passed until it has been run and its result checked. If the frontend is not connected end-to-end, record that limitation accurately.

CHAPTER 4 — APPLICATION ARCHITECTURE

4.4 Application-to-Database Flow

The intended architecture separates the user interface, application logic, database driver, and database server. MySQL Workbench is shown as an independent administration tool.



Workbench is a separate administration tool, not the application request path.

Request-response sequence

- The user enters information through the browser-based interface.
- Frontend JavaScript sends an HTTP request to an Express API endpoint.
- Express receives the request and validates input on the server.
- The backend uses mysql2 to execute parameterized SQL against MySQL Server.
- MySQL processes the query and returns rows or a status result.
- The backend returns an HTTP response and the frontend presents the result.

Security and reliability note

Use parameterized SQL queries rather than concatenating user input into SQL strings. Keep database credentials in environment configuration, avoid exposing them to the browser, validate data on the server, and handle database errors without leaking sensitive details. These are implementation recommendations; verify that each is present in the actual code.

Implementation status must be documented honestly. The project prompt specifies the intended technology stack, but no source code or API route list was supplied for independent verification.

CHAPTER 5 — CONCLUSION & REFERENCES

5.1 Conclusion

The Car Rental System is designed to demonstrate how a relational database can organize vehicle, customer, rental, and payment information. The project provides a practical context for explaining tables, primary keys, foreign keys, uniqueness constraints, SQL DDL and DML, joins, aggregation, and application-to-database communication. The final evaluation should be supported by the actual SQL script, interface screenshots, and verified query/test outputs.

5.2 Limitations to Review

• Confirm whether user authentication and administrator roles are implemented.

• Check whether overlapping bookings are prevented for the same vehicle.

• Verify how rental duration and total cost are calculated.

• Confirm late-return penalties, invoice generation, and payment integration only if present.

• Review error handling, input validation, transaction safety, and deployment configuration.

5.3 Future Improvements

• Role-based authentication and access control.

• Advanced vehicle search and availability filters.

• Booking-conflict prevention and safer concurrent updates.

• Automatic rental-cost calculation and late-return rules.

• Printable invoices, receipts, and rental/payment reports.

• Improved logging, error handling, secure deployment, and cloud hosting.

5.4 References

• AD Patel Institute of Technology, *DBMS Project Guide, Topics & Submission Guidelines*, supplied project guide, especially the project report structure and presentation outline.

• MySQL Documentation — SQL statements, data types, constraints, and relational database concepts: https://dev.mysql.com/doc/

• Node.js Documentation: https://nodejs.org/docs/

• Express.js Documentation: https://expressjs.com/

• mysql2 Node.js driver documentation / project repository: https://github.com/sidorares/node-mysql2

FINAL SUBMISSION CHECKLIST

• Fill group number, faculty name, academic year, and submission date.

• Confirm team roles and obtain required signatures.

• Replace illustrative schema/SQL names with the exact final SQL definitions.

APPENDIX — FINAL IMPLEMENTATION VERIFICATION

Use this page as a practical review sheet before submitting the report. The items below are intentionally phrased as checks because the final SQL script and source code were not supplied with the project details.

Area	Verification question	Status
Database	Is the database named CarRentalDB?	[VERIFY]
Tables	Are Vehicles, Customers, Rentals, and Payments the exact table names?	[VERIFY]
Columns	Do the field names and data types match the SQL script?	[VERIFY]
Keys	Are PK, FK, UNIQUE, and NOT NULL constraints present as described?	[VERIFY]
Cardinality	Does the schema allow the stated customer/rental/payment relationships?	[VERIFY]
Normalization	Have functional dependencies and candidate keys been reviewed?	[VERIFY]
Backend	Does Express use mysql2 and parameterized SQL?	[VERIFY]
Frontend	Are forms connected to working API endpoints?	[VERIFY]
Evidence	Are screenshots and test outputs from the actual implementation?	[VERIFY]

Important note

This report has been prepared from the project information in the supplied prompt and the college guidelines. Because no final SQL script, source code, or screenshots were provided, the database schema, SQL snippets, and test evidence sections are clearly marked for confirmation. Update these sections to match the actual project before academic submission.